

PRINCIPAL COMPONENT ANALYSIS (PCA)

I PCA LÀ GÌ? (HIỂU ĐÚNG BẢN CHẤT)

PCA (Principal Component Analysis) là một thuật toán Unsupervised Learning được sử dụng để **giảm số chiều dữ liệu** (Dimensionality Reduction).

- ✓ **Giảm số chiều** dữ liệu.
- ✓ **Vẫn giữ lại nhiều thông tin** (variance) nhất có thể.
- ✓ Bằng cách **tìm ra các trục mới tối ưu** (Principal Components) thay cho các trục gốc.

Ví dụ Trực giác: Nếu Toán và Lý có tương quan mạnh (học sinh giỏi Toán thường giỏi Lý), PCA sẽ xoay hệ trục tọa độ, gom hầu hết "thông tin quan trọng" vào **1** trục mới duy nhất (**PC1**), giảm từ 2D $\rightarrow$ 1D.

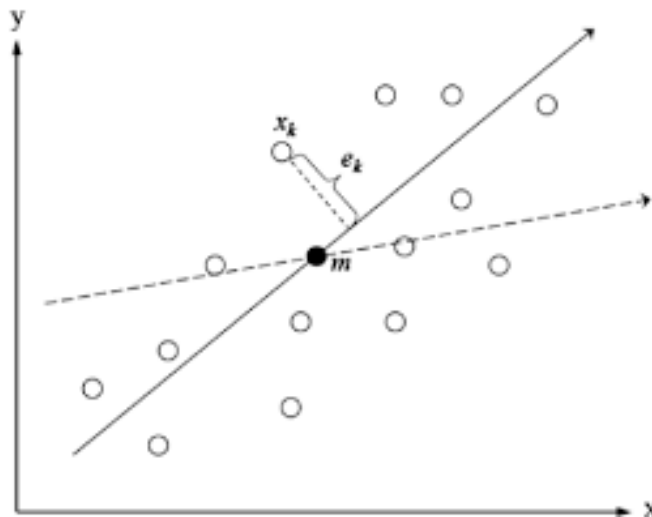
II PCA THỰC SỰ ĐANG TỐI ƯU CÁI GÌ?

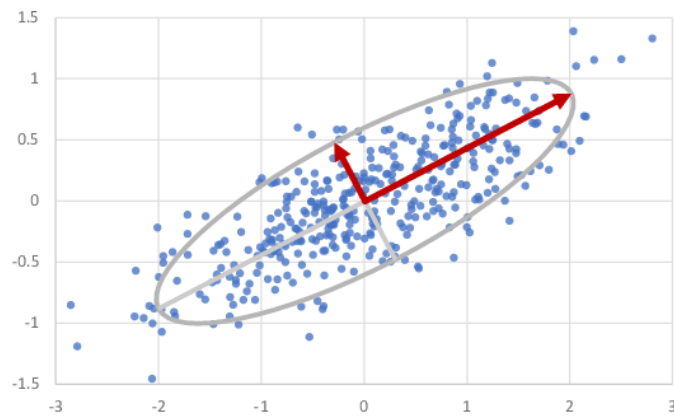
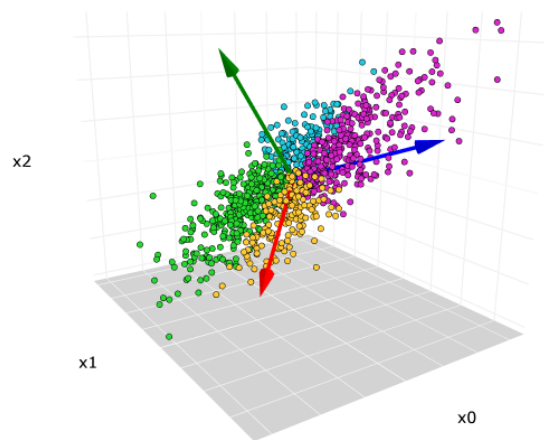
PCA tối ưu theo tiêu chí: **Tối đa hóa Phương sai** (Variance).

- **PC1** (Component 1): Hướng có **phương sai lớn nhất**.
- **PC2** (Component 2): Hướng có phương sai lớn thứ hai, và **vuông góc** với PC1.

$\rightarrow$ Các Principal Component (PC) là các vector **không trùng nhau**, mỗi vector cố gắng giữ nhiều "biến động" của dữ liệu nhất có thể.

III Trực giác hình học của PCA





Nhìn bằng hình học:

- Dữ liệu ban đầu nằm nghiêng trong mặt phẳng
- PCA xoay trục tọa độ
- PC1 = trục dài nhất của “đám mây dữ liệu”
- PC2 = trục ngắn hơn, vuông góc PC1
- Sau đó ta có thể chiếu toàn bộ điểm lên PC1

Thông tin vẫn giữ $\sim 95\text{--}99\%$, nhưng số chiều giảm mạnh.

IV PCA HOẠT ĐỘNG NHƯ THẾ NÀO?

Giả sử ta có tập dữ liệu X với m điểm và n đặc trưng.

BƯỚC 1: Chuẩn hóa dữ liệu (Standardization)

PCA rất nhạy với thang đo, vì vậy dữ liệu phải được **chuẩn hóa** (trừ Mean và chia Std) để các đặc trưng không bị ảnh hưởng bởi đơn vị hay độ lớn.

$$X_{\text{std}} = \frac{X - \mu}{\sigma}$$

Lưu ý: Nếu không chuẩn hóa, cột có giá trị lớn (ví dụ: Thu nhập) sẽ chi phối hoàn toàn kết quả của PCA.

BƯỚC 2: Tính ma trận hiệp phương sai (Covariance Matrix)

Ma trận hiệp phương sai (Σ) mô tả **phương sai** của từng biến (trên đường chéo chính) và **hiệp phương sai** giữa các cặp biến (các phần tử còn lại).

$$\Sigma_{ij} = \text{Cov}(X_i, X_j)$$

Ý nghĩa: $\text{Cov} \neq 0$ cho thấy 2 đặc trưng có liên hệ tuyến tính, và đây là thứ PCA muốn khai thác để giảm chiều.

BƯỚC 3: Tính Eigenvalue (λ) & Eigenvector (v)

Ta giải bài toán phương trình đặc trưng: $\Sigma v = \lambda v$.

- **Eigenvalue(λ)** : Lượng **phương sai** hay lượng **thông tin** mà PC tương ứng giữ lại được.
- **Eigenvector(v)** : **Hướng** của trục chính (Principal Component) trong không gian gốc.

Ví dụ: $\lambda_1 = 8.2$ và $\lambda_2 = 0.3$. λ_1 lớn hơn nhiều cho thấy PC1 giữ lại hầu hết thông tin.

BƯỚC 4: Sắp xếp theo độ quan trọng

- Sắp xếp các Eigenvalue giảm dần: $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$.
- **Chọn k thành phần đầu:** Dựa vào ngưỡng thông tin cần giữ lại (ví dụ: 95% Variance) để chọn k Eigenvector tương ứng.

BƯỚC 5: Tạo ma trận chiếu (Projection Matrix)

Tạo ma trận W bằng cách ghép k Eigenvector (Principal Component) đã chọn:

$$W = [v_1, v_2, \dots, v_k]$$

BƯỚC 6: Chiếu dữ liệu xuống không gian mới

Dữ liệu đã giảm chiều (Z) được tính bằng cách nhân dữ liệu chuẩn hóa với ma trận chiếu:

$$Z = X_{\text{std}} W$$

→ Z là dữ liệu sau PCA, có số chiều là k (với $k < n$).

VÍ DỤ

Ta xét ví dụ 5 sinh viên với 2 đặc trưng (Toán, Lý).

Sinh viên	Toán	Lý
1	2	3
2	3	3
3	4	4
4	5	5
5	6	5

Kết quả các bước trung gian

- Ma trận Hiệp phương sai: $\Sigma = \begin{pmatrix} 2.5 & 1.3 \\ 1.3 & 1.75 \end{pmatrix}$
- Eigenvalues: $\lambda_1 \approx 3.427$, $\lambda_2 \approx 0.073$
- Eigenvectors: $v_1 \approx [0.851, 0.526]$, $v_2 \approx [-0.526, 0.851]$

Chiều dữ liệu lên trục chính (PCs)

Chiều điểm đầu tiên $[-2, -1]$ lên PC1:

$$z_1 = [-2, -1] \cdot \begin{bmatrix} 0.851 \\ 0.526 \end{bmatrix} = -2 \cdot 0.851 + (-1) \cdot 0.526 \approx -2.228$$

Các điểm khác chiều tương tự:

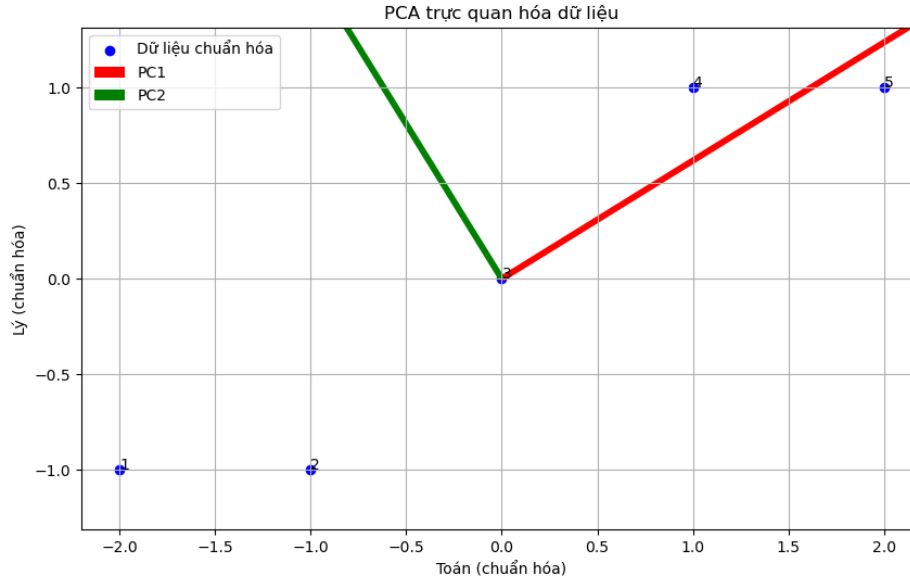
Sinh viên	PC1	PC2
1	-2.228	0.472
2	-1.377	0.052
3	0	0
4	1.377	-0.052
5	2.228	-0.472

Giảm chiều

- PC1 giữ phần lớn phương sai:

$$\frac{3.427}{3.427 + 0.073} \approx 97.9\%$$

- PC2 phương sai nhỏ $\Rightarrow$ có thể bỏ, giảm từ 2D $\rightarrow$ 1D
- Dữ liệu mới chỉ còn trục PC1, vẫn giữ hầu hết thông tin.



Đồ thị trực Oxy và PCA cho trực quan hóa dữ liệu

VI BẢN CHẤT TOÁN HỌC SÂU HƠN

PCA thực chất là bài toán tối ưu hóa đa mục tiêu:

1. Tối đa hóa Phương sai: Tìm hướng chiều mà dữ liệu được trải rộng nhất.
2. Ràng buộc Vector Vuông góc: Mỗi PC tiếp theo phải vuông góc với PC trước đó để đảm bảo không có sự chồng chéo thông tin.

→ Bài toán này **tương đương** với việc phân rã giá trị riêng (Eigen Decomposition) của ma trận hiệp phương sai.

VII PCA GIỮ THÔNG TIN THEO NGHĨA GÌ?

- ✓PCA chỉ giữ lại **Biến động** (Variance).
- X PCA **không đảm bảo** giữ được: Nhân (label), cấu trúc phân lớp, hoặc tính tách biệt của các cụm.

→ Vì PCA là Unsupervised, nó **không quan tâm** đến việc phân loại hay phân cụm.

VIII KHI NÀO NÊN DỪNG VÀ KHÔNG NÊN DỪNG PCA?

Nên dùng khi

- Dữ liệu **nhiều chiều** (≥ 10 đặc trưng).
- Các đặc trưng **có tương quan cao** (redundant).
- Mục tiêu là: Tăng tốc mô hình, Giảm overfitting, Nén dữ liệu, hoặc Visualize dữ liệu xuống 2D/3D.

KHÔNG nên dùng khi

- Dữ liệu **ít chiều** (không cần giảm).
- Các đặc trưng **độc lập** (PCA không hiệu quả).
- Cần **giải thích ý nghĩa gốc** của từng đặc trưng (vì PC là sự kết hợp tuyến tính của các biến gốc).
- Dữ liệu có nhiều biến **phân loại** (Categorical) (PCA dựa trên phương sai của dữ liệu số).

IX HIỂU SAI THƯỜNG GẶP VỀ PCA

- **Hiểu sai:** PCA là lựa chọn đặc trưng.
- **Thực tế:** PCA **tạo** đặc trưng mới (kết hợp tuyến tính các biến gốc).
- **Hiểu sai:** PCA giữ toàn bộ thông tin.
- **Thực tế:** PCA chỉ giữ **phương sai** và thường loại bỏ thông tin nhiễu.
- **Hiểu sai:** PCA luôn làm mô hình tốt lên.
- **Thực tế:** Nếu thông tin phân lớp nằm ở PC bị loại bỏ, PCA có thể làm giảm độ chính xác.